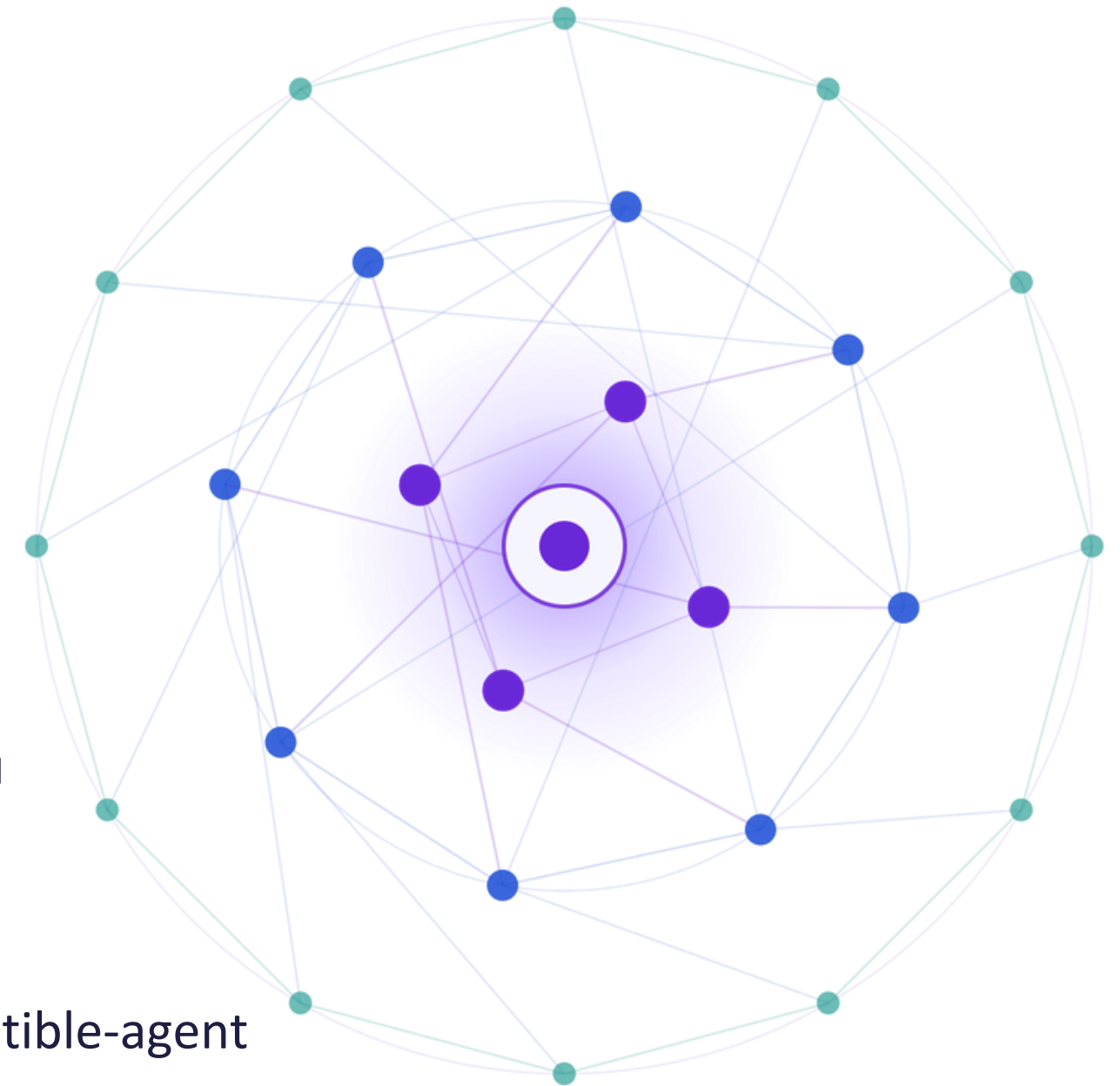


SAMSUNG PRISM

Generative AI Hackathon

3rd Edition 2026 – 27

- Theme ID - Theme 05
- Team Name - Team Astra
- College Name - Thapar Institute of Engineering and Technology
- Member Name & Email 1- Rishabh Saraswat | rsaraswat_be24@thapar.edu
- Member Name & Email 2- Koushik Kotha | kkotha_be24@thapar.edu
- Member Name & Email 3- Kasarapu Rajeev | krajeev_be24@thapar.edu
- Member Name & Email 4- Suneet Singh Arora | sarora2_be24@thapar.edu
- Submission Github link - <https://github.com/KasarapuRajeev/relay-interruptible-agent>



Theme 05 : Interruptible Real Time Agents

The core idea



One-line definition

A voice-native architecture for full-duplex conversation where users can interrupt, correct or re-plan mid-task.

The core problem

Real users change destinations, add constraints or cancel mid-execution. Perception, reasoning, tools and speech must share one consistent timeline.

What success looks like

- Fast interruption
- No duplicate actions
- No stale results
- Context preserved

Six technical objectives

1. Floor management

- Detect speech quickly
- Low latency
- No false completion claims

2. Interruption recovery

- Detect reliably
- Cancel superseded work
- Block stale speech/actions

3. Session slot tracking

- Keep context across turns
- Apply local corrections
- Keep slot values consistent

4. Schema-driven tools

- Parse tool schemas
- Use valid parameters
- Never duplicate state changes

5. Multimodal grounding

- Reason over audio/video
- Link multiple modalities
- Ground tool use in inputs

6. Protocol compliance

- Valid structured outputs
- Follow format/schema
- Safe + reliable responses

Evaluation

Task Completion

40%

Interruption Recovery

35%

Latency

15%

Safety & Protocol

10%

Existing Solutions & Gaps

Existing Solutions (voice-agent landscape, 2026)

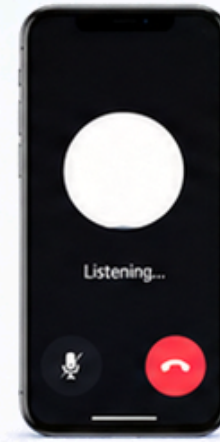
Gemini Live

- Sub-second latency, mid-sentence barge-in
- Now on Gemini 3.5 Live (better interruption handling)
- Improved background tool triggering



ChatGPT Advanced Voice Mode

- Native audio-to-audio pipeline (benchmark for barge-in)
- End-to-end audio processing (< 300 ms)
- Stops mid-token generation instantly



alexa+

- LLM-powered relaunch
- Past 1M+ registrations
- Voice-first by design



Open real-time agent frameworks

LiveKit



- Growing set of open real-time agent frameworks (LiveKit, GPT-Realtime, etc.) used to build voice agents

Key Strengths



Audio-level interruption

Stop speech generation instantly



Low latency

Expressive and natural speech synthesis



Acknowledges Instructions

Handles user corrections well



Increasingly natural

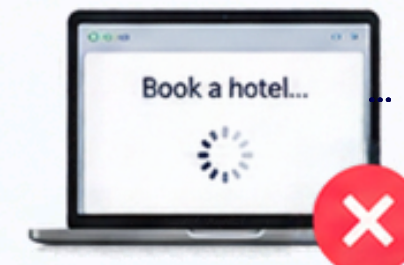
More human-like conversations

The Gap



No guarantees on cancelled actions

Results from cancelled work can still leak into the next answer



No safeguards against double-firing

A state-changing action can still execute after a mid-flow correction



No standard mechanism to pause and resume tasks

Have to recreate context and start over



Interruption handling inside the model layer

Not a portable, auditable control layer in front of any model

Our Solutions & Architecture Diagram

Our Model: Relay

Relay is a session scoped runtime that lets an AI agent be interrupted, corrected, cancelled, paused, and resumed while reasoning and tools are running without mixing stale work into the current answer.

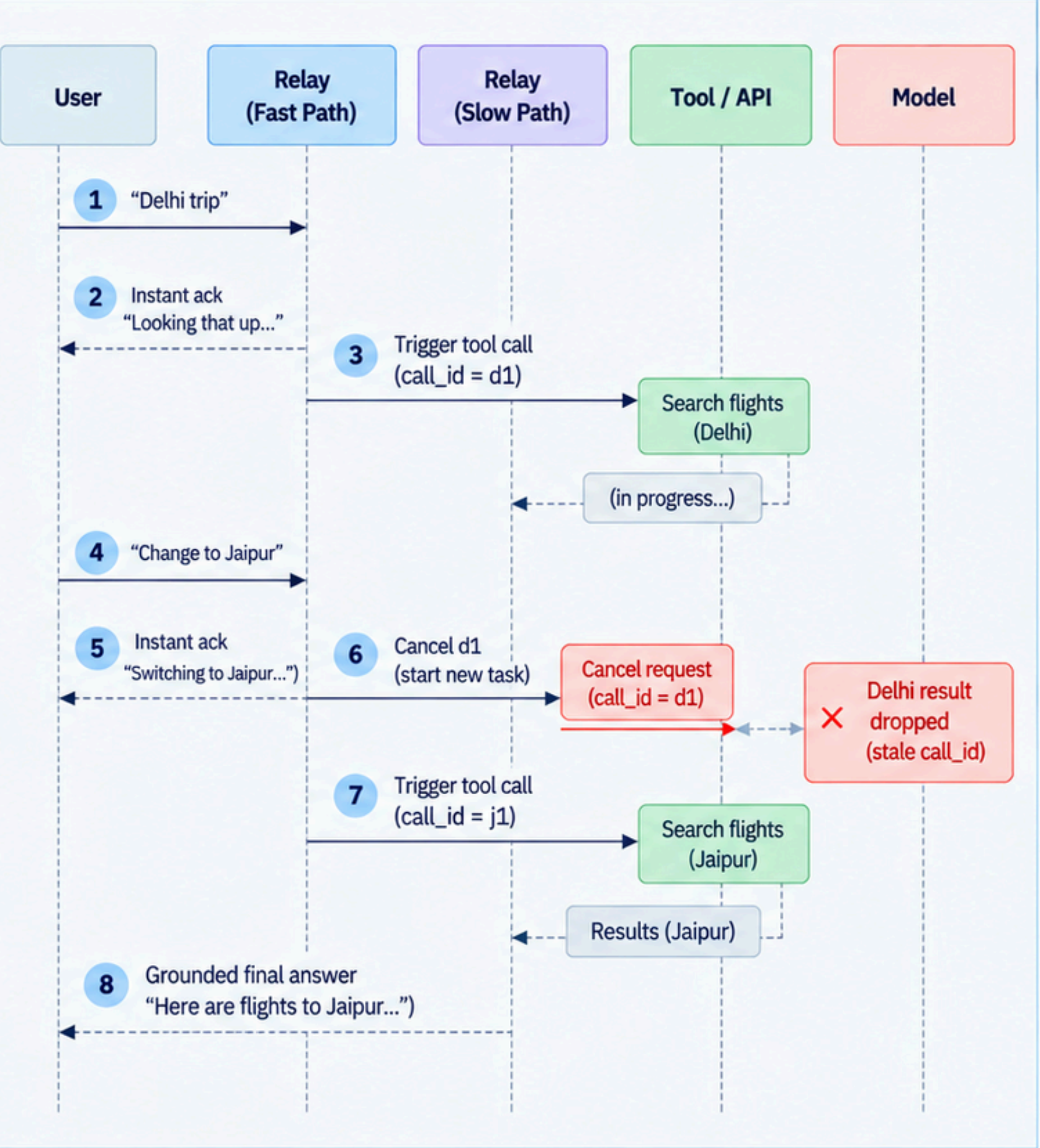
Scope and Responsibilities

Responsibilities of Relay	Responsibilities of the connected model
- Streaming events - Fast and slow execution - Intent, slot and task state - Tool validation - Cancellation - Stale result rejection - Exactly once side effects - Trace logging	- Language understanding and response generation - May be Gemini, OpenAI or an offline deterministic fallback via one ModelAdapter interface - Relay is not training a new model - It is the control layer around one

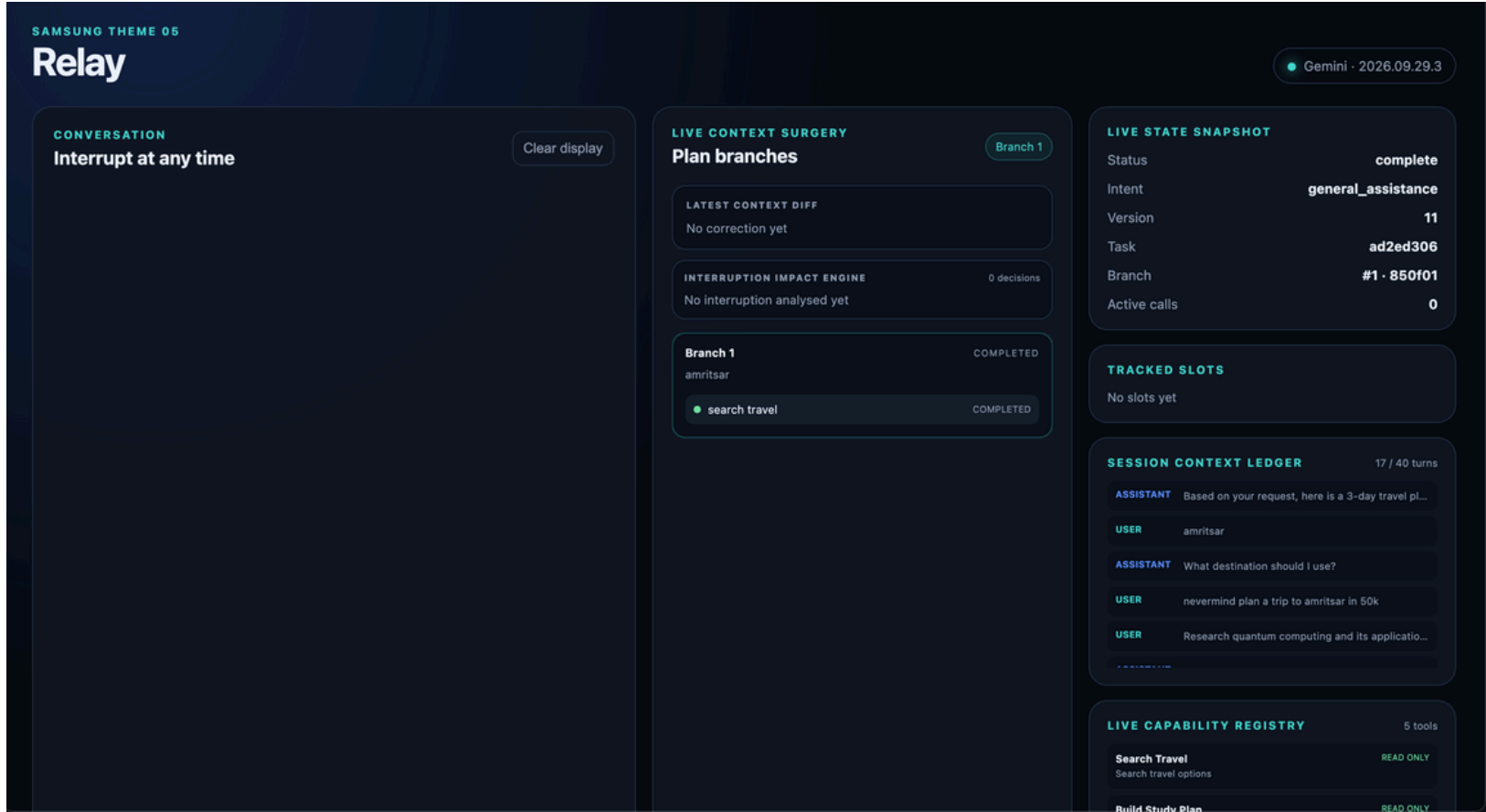
How it works

Fast path	Acknowledges instantly, before slow work starts.
Slow path	Runs tools and reasoning as a separately cancellable coroutine.
Coordination layer	- Unique call_id per tool invocation and explicit cancellation when superseded - Late results dropped once their call_id is no longer active - Idempotency keys block duplicate state changing calls - Versioned StateSnapshot (intent, slots, active or paused tasks) updated on every turn

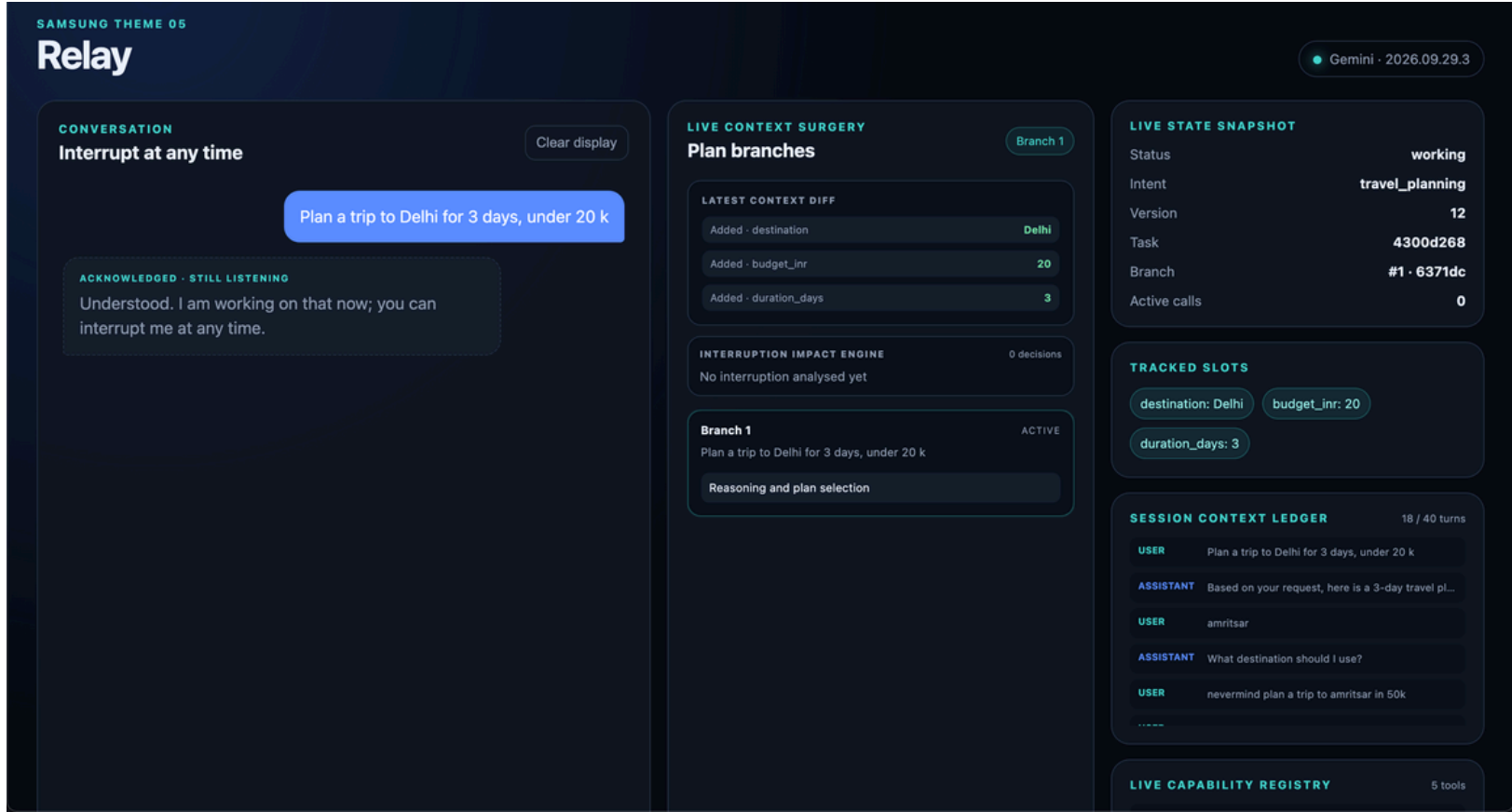
Interruption Flow (Sequence Diagram)



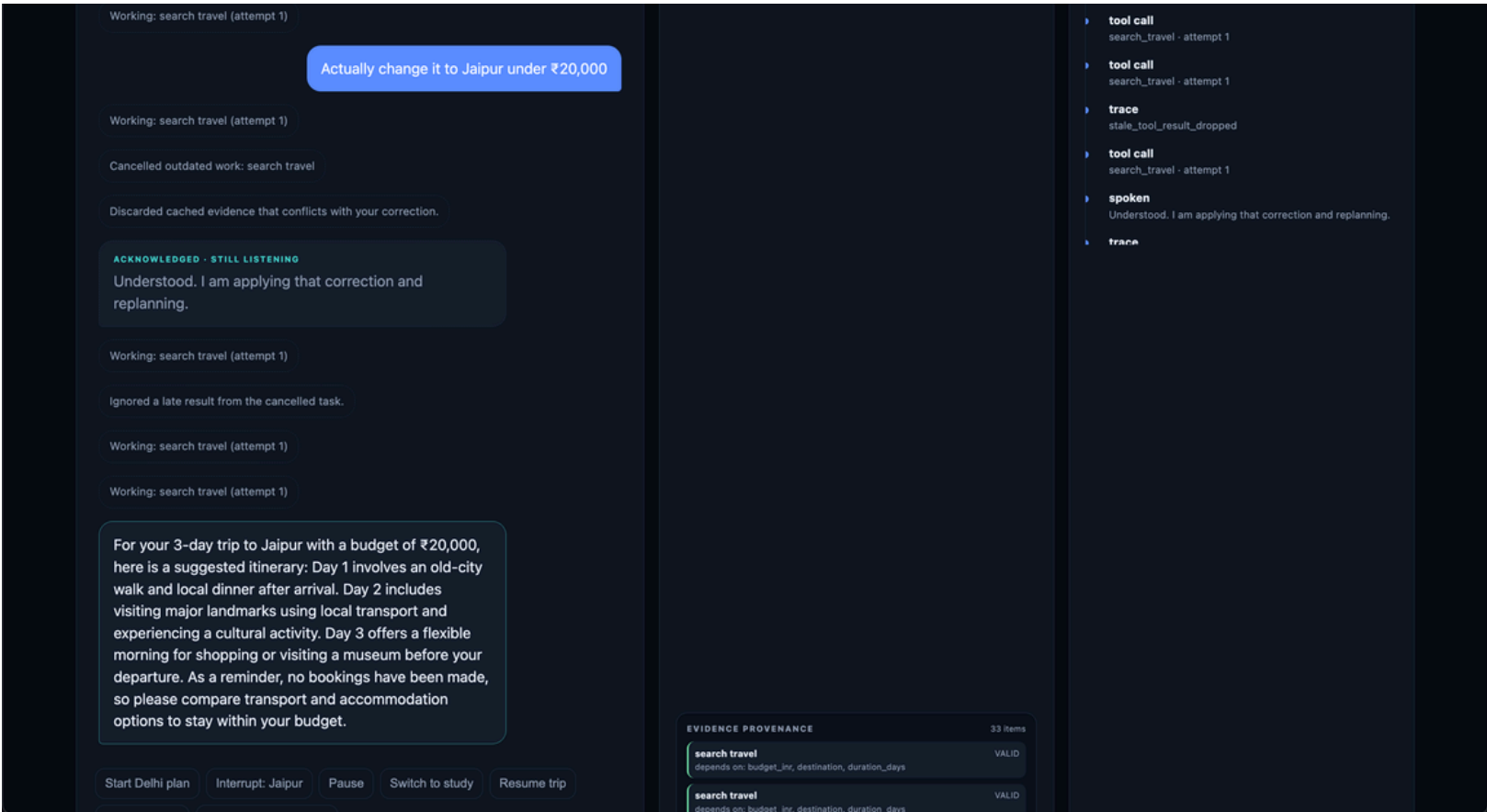
Demo & Product Walkthrough



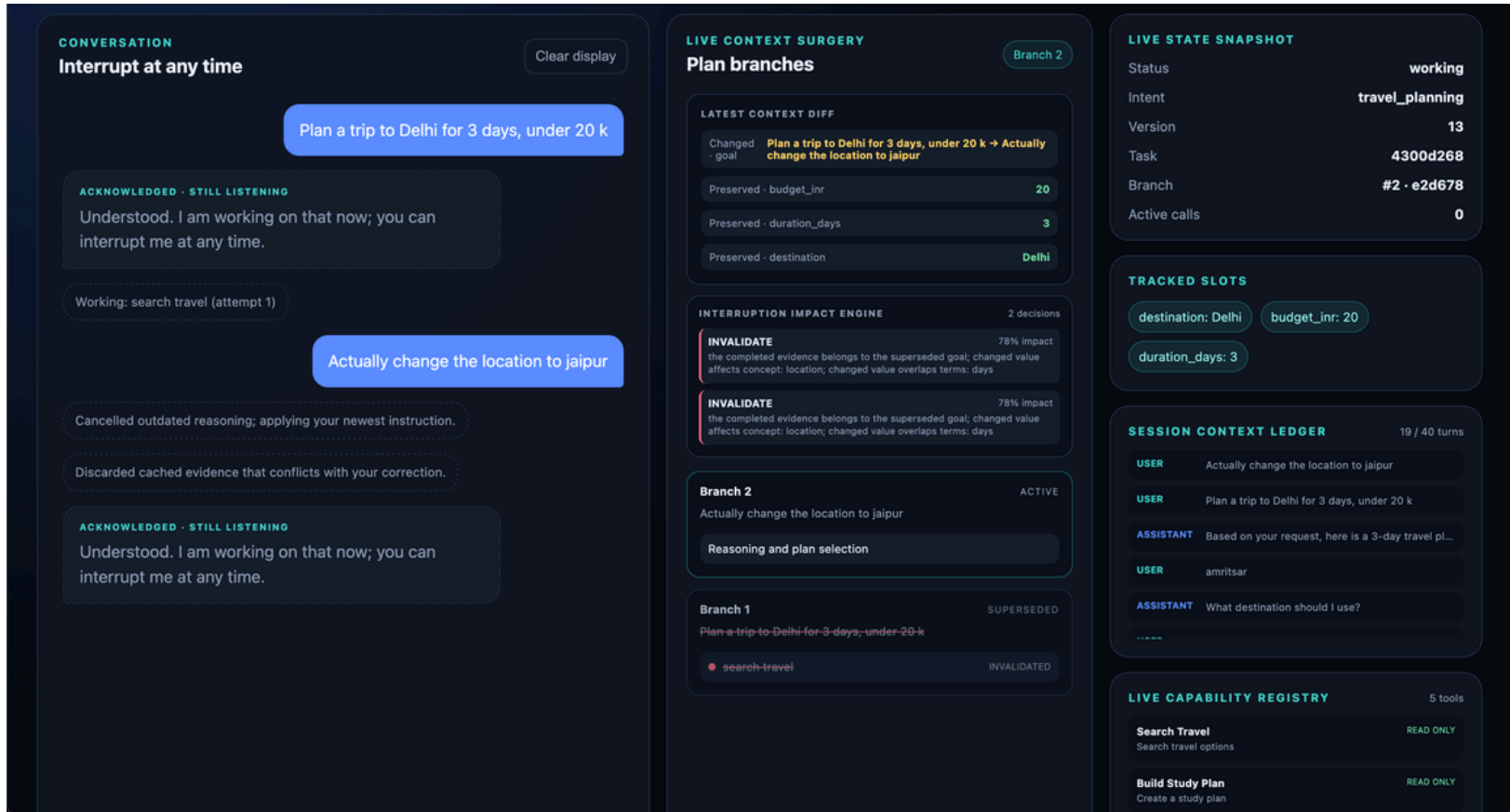
1. Dashboard



2. Give Initial Prompt



4. Output Recived wrt interrupt



3. Interrupt the prompt

Tools and tech stack used



Python



asyncio



Gemini



OpenAI



Git



HTML



CSS



JavaScript

Design Principle

Control layer, not a new model — Python + asyncio at the core, LLM behind a swappable adapter.

Layer	Technology	Why
Core runtime	Python 3.10 – 3.12, asyncio	Runs fast + slow paths concurrently
Event protocol	JSON events & actions	Structured, versioned, replayable
Tool layer	JSON Schema manifests	Validates args, blocks duplicates
Model layer	Gemini · OpenAI · offline adapter	Swap providers, zero orchestration change
Dashboard	Python HTTP, HTML/CSS/JS	Lightweight, zero dependencies
Testing	unittest, virtual-clock harness	32 tests, reproducible traces
Dev tooling	Git, PowerShell, env-based keys	Secure, single-instance startup

Key Takeaway

Python asyncio for orchestration, LLM-agnostic adapters, JSON-Schema tools for safe execution.

Planned

- Speech-to-text adapter for WAV input
- Vision adapter for PNG frames
- Full virtual-clock scheduling with fault injection

Impact and Use Case

Relay is a session-scoped runtime that lets an AI agent be interrupted, corrected, paused, and resumed — without stale work leaking into the current answer.

Impact

- **No duplicate actions.**
Retry a request and it still only happens once.
- **No dead air, no lying.**
The agent says where it actually is, instead of going silent or claiming it's done.
- **No zombie results.**
A cancelled search can't sneak back in and override the live one.
- **No lost progress.**
Pause a trip, plan a study session, come back — nothing's forgotten.

Other real-world use cases



Navigation
& travel



Customer
support



Bookings
& purchases



Education
& planning

Use case

Same conversation. Interrupt, switch, and resume — without losing context.



Innovation Highlights, Results and Limitations

Innovation Highlights

- 1 Live Context Surgery**
Three-panel view showing preserved, changed, and replanned state.
- 2 Provider-independent Model Adapter**
One adapter interface for Gemini, OpenAI, or an offline model.
- 3 Exactly-once safety for mutations**
Idempotency keys prevent duplicate calls.
- 4 Fast/slow dual-path execution**
Immediate acknowledgement while slow tool execution runs concurrently.
- 5 Session task checkpointing**
Pause, switch, and resume tasks with slots restored.

Key Results

- 1 32/32 automated tests passing**
Across slot extraction, corrections, cancellation, replanning, retries, and model decisions.
- 2 Verified end-to-end chain (35-event trace)**
Live Gemini run for travel and study scenarios, including interruption → correction → real result.
(Spoken acknowledgement → Tool call → Grounded result → Final response)
- 3 Cached-result validation verified**
Delhi plan correctly invalidated when corrected to Jaipur.
- 4 Resilience result**
Gemini 503s handled with two bounded retries + model fallback.

Limitations

- 1 Deterministic parsing**
Intent/slot parsing is currently deterministic, not yet fully model-powered.
- 2 Requires API keys**
Live provider calls need the user's own API key/quota; Claude adapter not yet implemented.
- 3 Mock data sources**
Travel and study tools use mock manifests/injected results (no live data source yet).
- 4 Multimodal not supported**
Audio/image inputs are acknowledged but not yet decoded or grounded.

What's next

Technical features to add

Complete core capabilities and extend Relay's ecosystem.

Full propose/commit/reconcile lifecycle

Staged lifecycle to hold, validate and commit actions for duplicate-action safety.

Persistent cross-session memory

Currently session-scoped only. A durable store lets a paused task survive a full session end.

Multimodal grounding, completed

WAV transcription and PNG vision adapters are designed but not wired in.

Real data integrations

Replace manifest-based mock tools with real flight, hotel and booking APIs with live pricing.

Tool marketplace / plugin registration

Let third-party tools register into the existing JSON-Schema manifest at runtime.

Deployment path

Steps to ship a production-ready version.

Containerize the runtime

Package Relay and dashboard as a Docker image for reproducible single-instance startup.

Move session state out of memory

Use a Redis-backed session store to support multiple instances behind a load balancer.

Replace dashboard polling with WebSockets/SSE

Needed for real-time state updates at scale.

Add authentication

Per-user API keys or OAuth before this touches user data.

Secrets management

Move from environment variables to a proper secrets manager (AWS Secrets Manager, Vault).

CI/CD pipeline

Run the existing test suite on every commit, plus cross-version testing.

Observability

Send structured trace logs to a monitoring stack (Grafana, Datadog).

Rate limiting and quota management

Protect against provider rate limits at the account level, not just per request.

Brownie points slide (differentiation)

Most agent demos hide interruption handling in a black box. Relay makes it visible by showing **every preserved, changed, cancelled and replanned piece of work on screen in real time.**

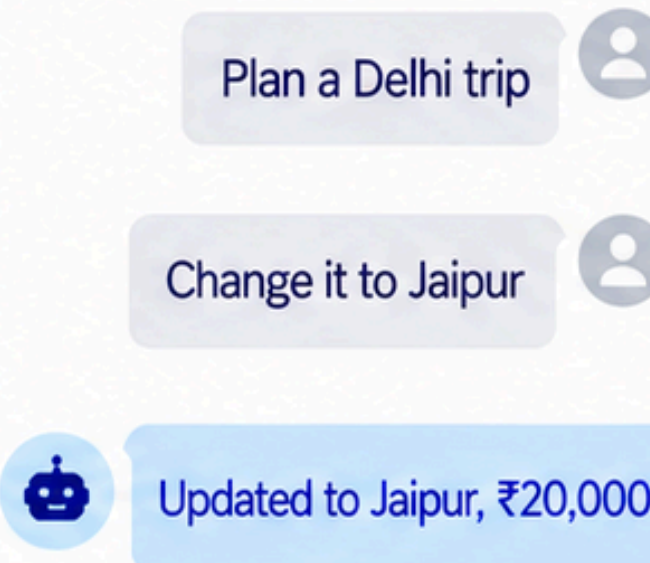
Live Context Surgery

The signature capability.

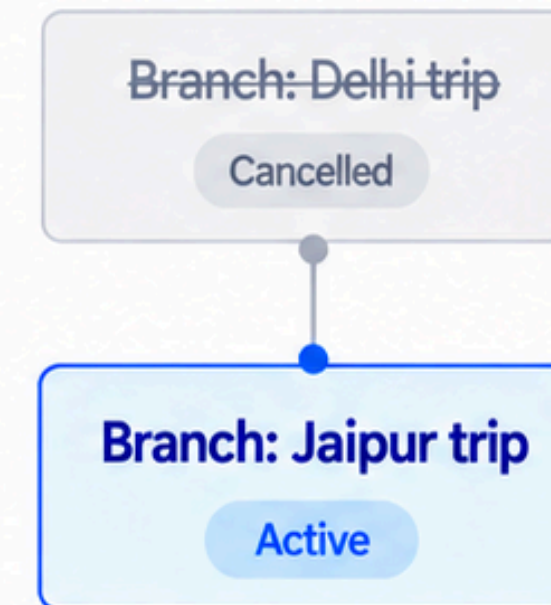
- 1 Three-panel view**
Conversation, live branch/plan graph and agent inspector.
- 2 Audit trail, not deletion**
Obsolete work gets struck through and stays visible.
- 3 Traceable and selective invalidation**
Every branch has an ID and each step traces back to the constraint that created it, enabling selective invalidation instead of scrapping the whole plan.

Example: Delhi → Jaipur correction (live)

Conversation



Live plan graph



Agent inspector

Destination	Jaipur
Budget	₹20,000
Cancelled call	delhi_search
Active call	jaipur_search

Why this is the brownie-points moment, not just a feature



A judge can **watch the Delhi to Jaipur correction happen** and see the old branch struck through, the new one spawn and the untouched parts stay untouched instead of just taking our word for it.

Checklist- Updated on Public GitHub

- Working prototype code — public or shared GitHub repo : Yes
- README with reproducible setup instructions : Yes
- Demo video, max 5 minutes : Yes
- Presentation file (PDF) : Yes

Thank you

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India